

Taller Práctico: Implementación, Análisis y Diagnóstico de Redes Neuronales Artificiales

Modelado y Simulación de Redes Neuronales

Fecha: 20 de septiembre de 2026

Ejercicio 1: Maximización de Función

Enunciado: Maximizar la función matemática $f(x) = x \sin(10\pi x) + 1$, evaluada en el intervalo $x \in [0, 1]$.

Explicación del código y funcionamiento: El código implementa un Algoritmo Genético (AG) clásico enfocado en la optimización numérica. Se inicializa una población de 50 cromosomas, donde cada individuo es una cadena binaria de 16 bits que representa una posible solución.

El funcionamiento se divide en los siguientes pasos iterativos a lo largo de 100 generaciones:

1. **Decodificación:** Convierte la cadena binaria de 16 bits a un número real continuo dentro del dominio $[0, 1]$.
2. **Evaluación de Aptitud (Fitness):** Evalúa el valor decodificado en la función matemática. Los valores más altos de $f(x)$ otorgan mayor aptitud al individuo.
3. **Selección:** Utiliza el método de "ruleta", dando mayor probabilidad de reproducción a los cromosomas con mejores resultados.
4. **Cruce y Mutación:** Combina pares de cromosomas (probabilidad del 80%) intercambiando segmentos de bits, y aplica una mutación ligera (5%) invirtiendo bits aleatorios para mantener la diversidad genética y evitar caer en óptimos locales. Finalmente, grafica la convergencia hacia el máximo global.

```
import numpy as np
import matplotlib.pyplot as plt
import random

# 1. Definición del problema y parámetros
POBLACION_SIZE = 50
CROMOSOMA_LEN = 16
GENERACIONES = 100
PROB_CRUCE = 0.8
PROB_MUTACION = 0.05

# 2. Función de Aptitud
def aptitud(x):
    return x * np.sin(10 * np.pi * x) + 1

# Decodificación de binario a real en el rango [0, 1]
def decodificar(cromosoma):
    valor_decimal = int("".join(str(i) for i in cromosoma), 2)
    max_decimal = (2**CROMOSOMA_LEN) - 1
    return valor_decimal / max_decimal

# 3. Operadores Genéticos
def crear_poblacion(size, length):
    return [[random.randint(0, 1) for _ in range(length)] for _ in range(size)]
```

```
def seleccion_ruleta(poblacion, aptitudes):
    suma_aptitud = sum(aptitudes)
    probabilidades = [apt / suma_aptitud for apt in aptitudes]
    seleccionados = random.choices(poblacion, weights=probabilidades, k=len(poblacion))
    return seleccionados

def cruce(padre1, padre2):
    if random.random() < PROB_CRUCE:
        punto = random.randint(1, CROMOSOMA_LEN - 1)
        hijo1 = padre1[:punto] + padre2[punto:]
        hijo2 = padre2[:punto] + padre1[punto:]
        return hijo1, hijo2
    return padre1, padre2

def mutacion(cromosoma):
    for i in range(CROMOSOMA_LEN):
        if random.random() < PROB_MUTACION:
            cromosoma[i] = 1 - cromosoma[i]
    return cromosoma

# 4. Ciclo Principal del AG
poblacion = crear_poblacion(POBLACION_SIZE, CROMOSOMA_LEN)
mejores_aptitudes = []

for gen in range(GENERACIONES):
    valores_x = [decodificar(ind) for ind in poblacion]
    aptitudes = [aptitud(x) for x in valores_x]

    mejor_aptitud_gen = max(aptitudes)
    mejores_aptitudes.append(mejor_aptitud_gen)

    poblacion_seleccionada = seleccion_ruleta(poblacion, aptitudes)

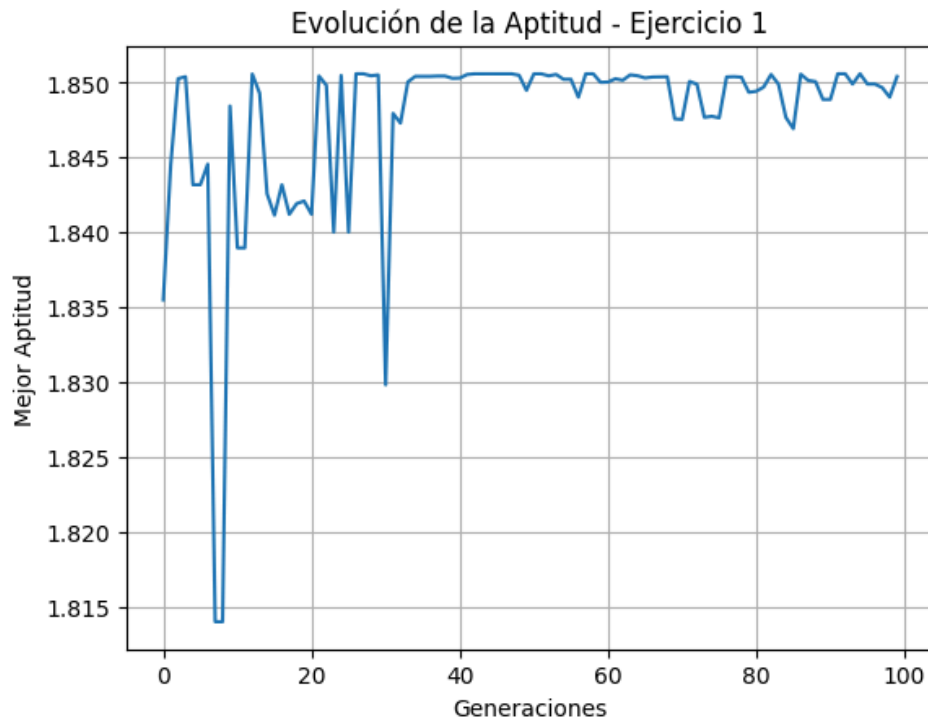
    nueva_poblacion = []
    for i in range(0, POBLACION_SIZE, 2):
        hijo1, hijo2 = cruce(poblacion_seleccionada[i], poblacion_seleccionada[i+1])
        nueva_poblacion.extend([mutacion(hijo1), mutacion(hijo2)])

    poblacion = nueva_poblacion

# Resultados
mejor_individuo = max(poblacion, key=lambda ind: aptitud(decodificar(ind)))
mejor_x = decodificar(mejor_individuo)
print(f"Mejor x encontrado: {mejor_x:.4f}")
print(f"Valor máximo f(x): {aptitud(mejor_x):.4f}")

plt.plot(mejores_aptitudes)
plt.title("Evolución de la Aptitud - Ejercicio 1")
plt.xlabel("Generaciones")
plt.ylabel("Mejor Aptitud")
plt.grid(True)
plt.show()
```

Mejor x encontrado: 0.8523
 Valor máximo $f(x)$: 1.8501



✓ Ejercicio 2: Verdadera Democracia (Asignación de Poder)

Enunciado: En un congreso conformado por 5 partidos políticos, se repartieron 50 curules de forma no uniforme. Se dispone de 50 entidades del gobierno, cada una con un "peso de poder" aleatorio (de 1 a 100). El objetivo es asignar estas entidades a los partidos para que la porción de poder total que recibe cada uno sea lo más equitativa y proporcional posible a su número de curules.

Explicación del código y funcionamiento: Este problema aborda la optimización combinatoria (similar al problema de la mochila). El algoritmo crea individuos representados por vectores de 50 posiciones (las entidades), donde cada gen contiene un valor entero del 0 al 4 (el partido asignado).

1. **Aptitud:** Calcula el error cuadrático medio entre el porcentaje de poder que *debería* tener un partido (según sus curules) y el porcentaje de poder real que suma con las entidades asignadas. La función busca maximizar el inverso de este error (minimizar la diferencia).
2. **Selección por Torneo:** En lugar de ruleta, escoge aleatoriamente 3 individuos y selecciona al mejor, aumentando la presión selectiva.
3. **Evolución:** Cruza los vectores intercambiando las asignaciones de entidades a partir de un punto aleatorio, y muta asignando ocasionalmente (2% de probabilidad) una entidad a un partido distinto. El proceso itera 200 generaciones buscando la distribución de poder más justa.

```
import numpy as np
import random
import matplotlib.pyplot as plt

# 1. Parámetros del escenario
NUM_PARTIDOS = 5
NUM_CURULES = 50
NUM_ENTIDADES = 50

curules_partidos = np.random.multinomial(NUM_CURULES, np.ones(NUM_PARTIDOS)/NUM_PARTIDOS)
```

```

porcentaje_poder_objetivo = curules_partidos / NUM_CURULES
pesos_entidades = np.random.randint(1, 101, NUM_ENTIDADES)
poder_total_entidades = sum(pesos_entidades)

# 2. Representación
POB_SIZE = 100
GENS = 200

# 3. Función de Aptitud
def calcular_aptitud(cromosoma):
    poder_asignado = np.zeros(NUM_PARTIDOS)
    for i, partido in enumerate(cromosoma):
        poder_asignado[partido] += pesos_entidades[i]

    porcentaje_asignado = poder_asignado / poder_total_entidades
    error = np.sum((porcentaje_asignado - porcentaje_poder_objetivo)**2)
    return 1 / (1 + error)

# 4. AG Principal
poblacion = [np.random.randint(0, NUM_PARTIDOS, NUM_ENTIDADES).tolist() for _ in range(POB_SIZE)]
historial_aptitud = []

for gen in range(GENS):
    aptitudes = [calcular_aptitud(ind) for ind in poblacion]
    historial_aptitud.append(max(aptitudes))

    nueva_poblacion = []
    for _ in range(POB_SIZE):
        torneo = random.sample(list(zip(poblacion, aptitudes)), 3)
        ganador = max(torneo, key=lambda x: x[1])[0]
        nueva_poblacion.append(ganador.copy())

    for i in range(0, POB_SIZE, 2):
        if random.random() < 0.7:
            punto = random.randint(1, NUM_ENTIDADES-1)
            temp = nueva_poblacion[i][punto:].copy()
            nueva_poblacion[i][punto:] = nueva_poblacion[i+1][punto:]
            nueva_poblacion[i+1][punto:] = temp

    for i in range(POB_SIZE):
        for j in range(NUM_ENTIDADES):
            if random.random() < 0.02:
                nueva_poblacion[i][j] = random.randint(0, NUM_PARTIDOS-1)

    poblacion = nueva_poblacion

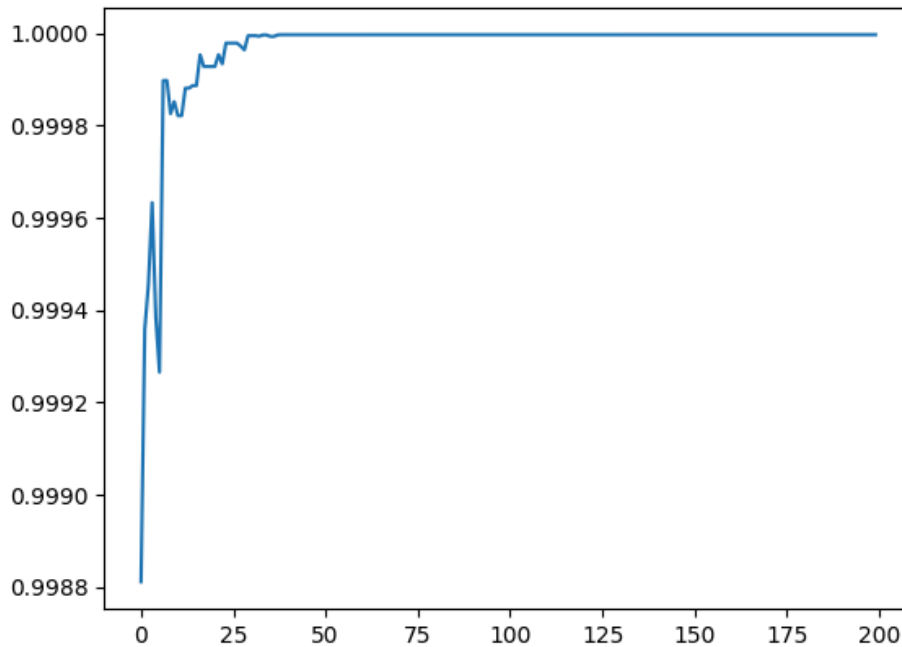
mejor_asignacion = max(poblacion, key=calcular_aptitud)
print("Distribución Objetivo de Curules:", curules_partidos)
poder_final = np.zeros(NUM_PARTIDOS)
for i, partido in enumerate(mejor_asignacion):
    poder_final[partido] += pesos_entidades[i]
print("Porcentaje de Poder Objetivo: ", np.round(porcentaje_poder_objetivo*100, 2))
print("Porcentaje de Poder Alcanzado: ", np.round((poder_final/poder_total_entidades)*100, 2))

plt.plot(historial_aptitud)
plt.title("Evolución de Asignación de Poder - Ejercicio 2")
plt.show()

```

Distribución Objetivo de Curules: [14 6 8 13 9]
 Porcentaje de Poder Objetivo: [28. 12. 16. 26. 18.]
 Porcentaje de Poder Alcanzado: [28.12 11.91 15.94 26.05 17.98]

Evolución de Asignación de Poder - Ejercicio 2



✓ Ejercicio 3: Problema del Despacho de Energía

Enunciado: Una empresa dispone de cuatro plantas generadoras con capacidades máximas de 3, 6, 5 y 4 GW para satisfacer la demanda de cuatro ciudades (Cali: 4 GW, Bogotá: 3 GW, Medellín: 5 GW, Barranquilla: 3 GW). Considerando una matriz de costos de transporte entre cada planta y ciudad, además de costos de generación por kW-h, se debe encontrar el despacho óptimo que minimice el costo total.

Explicación del código y funcionamiento: Se trata de un problema complejo de logística y transporte con múltiples restricciones operativas. La representación del cromosoma es una matriz de 4x4 (aplanada a un vector de 16 posiciones) que indica la cantidad de GW enviados desde la planta i a la ciudad j .

1. **Generación Inicial Inteligente:** Crea individuos garantizando que la demanda de las ciudades se satisfaga inicialmente con despachos de al menos 1 GW por iteración, evitando bucles infinitos y acelerando la convergencia.
2. **Penalizaciones:** La función de aptitud es altamente restrictiva. Calcula el costo económico real (transporte + generación), pero aplica multas matemáticas masivas (5000 a 10000 de peso) si una planta excede su capacidad máxima o si una ciudad no recibe exactamente la energía que demanda.
3. **Evolución con Elitismo:** El algoritmo guarda automáticamente al mejor individuo de cada generación (elitismo) para no perder la mejor solución logística descubierta. Aplica selección por torneos de 4 participantes y muta sumando o restando 1 GW en rutas aleatorias (asegurándose de no bajar de 0 GW). Tras 300 generaciones, entrega la matriz de envíos más económica.

```
import numpy as np
import random
import matplotlib.pyplot as plt

# 1. Definición del problema
capacidades = np.array([3, 6, 5, 4])
demandas = np.array([4, 3, 5, 3])
```

```

costos_transporte = np.array([
    [1, 4, 3, 6],
    [4, 1, 4, 5],
    [3, 4, 1, 4],
    [6, 5, 4, 1]
])
costos_gen = np.array([680, 720, 660, 750])

# 2. Representación
POB_SIZE = 150
GENS = 300

def crear_individuo():
    ind = np.zeros((4, 4), dtype=int)
    for j in range(4):
        restante = demandas[j]
        while restante > 0:
            i = random.randint(0, 3)
            # Corrección aplicada: Evitar bucle infinito cambiando el límite inferior a 1
            aporte = random.randint(1, restante)
            ind[i][j] += aporte
            restante -= aporte
    return ind.flatten().tolist()

# 3. Función de Aptitud
def aptitud_energia(cromosoma):
    matriz = np.array(cromosoma).reshape(4, 4)
    costo_total = 0
    penalizacion = 0

    gen_por_planta = np.sum(matriz, axis=1)
    recibido_por_ciudad = np.sum(matriz, axis=0)

    for i in range(4):
        if gen_por_planta[i] > capacidades[i]:
            penalizacion += (gen_por_planta[i] - capacidades[i]) * 5000

    for j in range(4):
        if recibido_por_ciudad[j] != demandas[j]:
            penalizacion += abs(recibido_por_ciudad[j] - demandas[j]) * 10000

    for i in range(4):
        for j in range(4):
            costo_total += matriz[i][j] * (costos_transporte[i][j] + costos_gen[i])

    return 1.0 / (costo_total + penalizacion + 1)

# 4. Ejecución del Algoritmo
poblacion = [crear_individuo() for _ in range(POB_SIZE)]
historial_costos = []

for gen in range(GENS):
    aptitudes = [aptitud_energia(ind) for ind in poblacion]
    mejor_ind = poblacion[np.argmax(aptitudes)]

    mejor_matriz = np.array(mejor_ind).reshape(4, 4)
    costo_real = sum(mejor_matriz[i][j] * (costos_transporte[i][j] + costos_gen[i])
                    for i in range(4) for j in range(4))
    historial_costos.append(costo_real)

    nueva_poblacion = [mejor_ind.copy()]
    for _ in range(POB_SIZE - 1):
        torneo = random.sample(list(zip(poblacion, aptitudes)), 4)

```

```

ganador = max(torneo, key=lambda x: x[1])[0]
nueva_poblacion.append(ganador.copy())

# Cruce
for i in range(1, POB_SIZE, 2):
    if i+1 < POB_SIZE and random.random() < 0.8:
        punto = random.randint(1, 14)
        temp = nueva_poblacion[i][punto:].copy()
        nueva_poblacion[i][punto:] = nueva_poblacion[i+1][punto:]
        nueva_poblacion[i+1][punto:] = temp

# Corrección aplicada: Mutación desindentada y aplicada correctamente
for i in range(1, POB_SIZE):
    for j in range(16):
        if random.random() < 0.05:
            nueva_poblacion[i][j] = max(0, nueva_poblacion[i][j] + random.choice([-1, 1]))

poblacion = nueva_poblacion

mejor_solucion = np.array(max(poblacion, key=aptitud_energia)).reshape(4, 4)
print("Mejor despacho de energía encontrado (Filas=Plantas, Cols=Ciudades):\n", mejor_solucion)
print("Costos totales (Mínimos alcanzados): $", historial_costos[-1])

plt.plot(historial_costos)
plt.title("Minimización de Costos de Transporte y Generación")
plt.ylabel("Costo Total")
plt.xlabel("Generaciones")

```

Mejor despacho de energía encontrado (Filas=Plantas, Cols=Ciudades):

```

[[1 0 0 2]
 [1 2 3 0]
 [1 1 2 1]
 [1 0 0 0]]

```

Costos totales (Mínimos alcanzados): \$ 10460

